


**PONTIFICIA UNIVERSIDAD JAVERIANA**
*Facultad de Ingeniería — Introducción a Sistemas Distribuidos*

# Diseño del Sistema Reservas de Boletos

**Integrantes:**
Elier David Ibarra Medina
Samuel Adrian Montaña Linares

**Profesor: Rafael Vicente Paez Mendez**

# Introducción

El Sistema Reserva de Boletos es una aplicación distribuida que permite a los estudiantes de la universidad consultar eventos culturales y deportivos disponibles en un mes determinado, reservar puestos y modificar sus reservas cambiando el número de puestos o la fecha si el evento ocurre en el mismo mes. El diseño está implementado en 3 capas, Clientes, Lógica de negocio y de Almacenamiento, distribuidas en 3 nodos donde su comunicación es por medio de una librería llamada ZeroMQ, combinando la sincronía para consultas y reservas y, la asíncronía para cancelaciones.

Para la primera entrega, se requieren las operaciones síncronas de consulta de eventos y reserva de puestos funcionando de extremo a extremo entre las 3 capas. El diseño completo de la arquitectura mediante los 4 modelos de sistema y los diagramas de UML. Y por último, la definición del protocolo de pruebas, la estratgia de recolección de métricas y el mecanismo de generación de carga para la evaluación de desempeño.

# Modelos del Sistema

## Modelo Físico

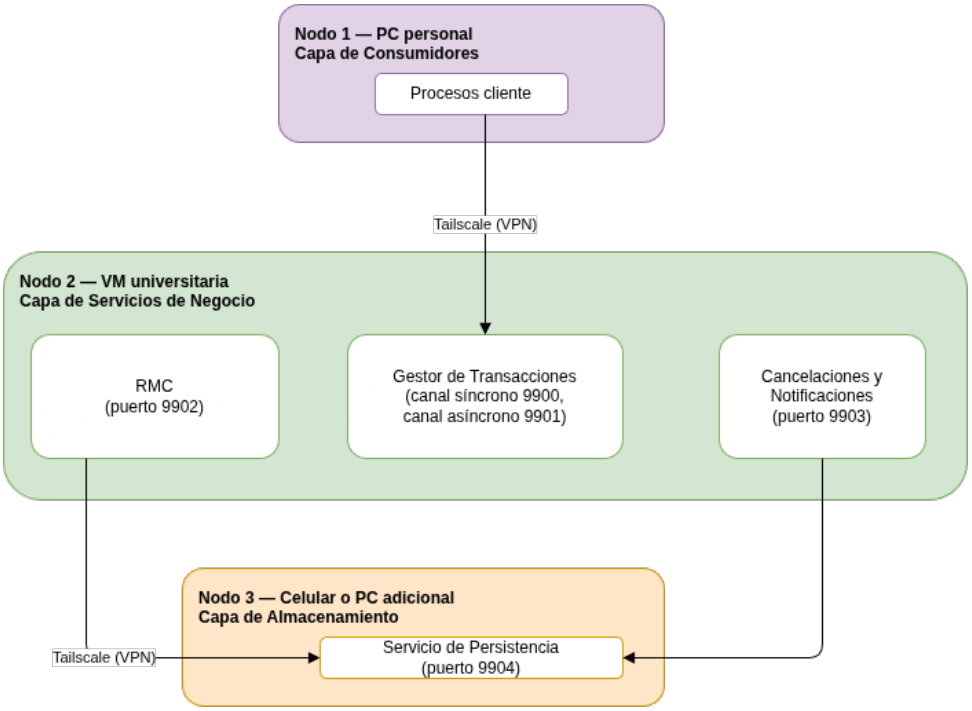



**Diagrama de Despliegue**

El sistema se implementa en el lenguaje de programación Go (versión 1.27.1), utilizando la librería ZeroMQ para Go `go-zeromq/zmq4` (versión 0.17.0), elegida por su soporte nativo de concurrencia mediante *goroutines* . Esto es importante, ya que para que el Gestor de Transacciones atienda múltiples solicitudes de forma eficiente. Su rendimiento como lenguaje compilado, es relevante para las métricas de desempeño evaluadas en este proyecto y por ser una implementación pura en Go, sin dependencias externas de C, lo que simplifica el despliegue en los tres nodos.

Como muestra el diagrama de despliegue, el sistema corre en 3 máquinas distintas, una por cada capa de la arquitectura:

- **Nodo 1** (Marca: Lenovo |Modelo: ThinkPad E16 Gen2 |SO: Linux |Distro: Mint 22.3 |RAM: 16Gb |Espacio: 512Gb): Se encarga de ejecutar los procesos cliente, ya que son los que generan las solicitudes de consulta, reserva y cancelación.
- **Nodo 2** (Marca: Dell |Modelo: XPS 13 9310 |SO: Linux |Distro: Mint 22.1 |RAM: 15Gb |Espacio: 229Gb): Se encarga de ejecutar los 3 servicios de la capa de negocio, el gestor de transacciones, el servicio RMC y el gestor de cancelaciones.
- **Nodo 3** (Marca: Google |Modelo: Pixel 9a |SO: Android 17 |Distro: Android |RAM: 8Gb |Espacio: 128Gb): Se encarga de ejecutar el servicio de persistencia, encargado de guardar y consultar los datos.

Como estos dispositivos no están en la misma red física, se usa **Tailscale** para crear una red privada virtual que les asigna direcciones IP fijas y los conecta entre sí sin importar dónde estén conectados. Cada servicio tiene un puerto fijo asignado, de tal forma que la comunicación sea siempre predecible. A continuación se muestra en la tabla los componentes del software y sus puertos de comunicación:

| **Componente** | **Nodo** | **Puerto** |
|---|---|---|
| Procesos Cliente | Nodo 1 | (se conecta al puerto 9900 del Nodo 2) |
| Gestor de Transacciones (canal síncrono) | Nodo 2 | 9900 |
| Gestor de Transacciones (canal asíncrono) | Nodo 2 | 9901 |
| Servicio RMC | Nodo 2 | 9902 |
| Servicio de Cancelaciones y Notificaciones | Nodo 2 | 9903 |
| Servicio de Persistencia | Nodo 3 | 9904 |

## Modelo de Interacción

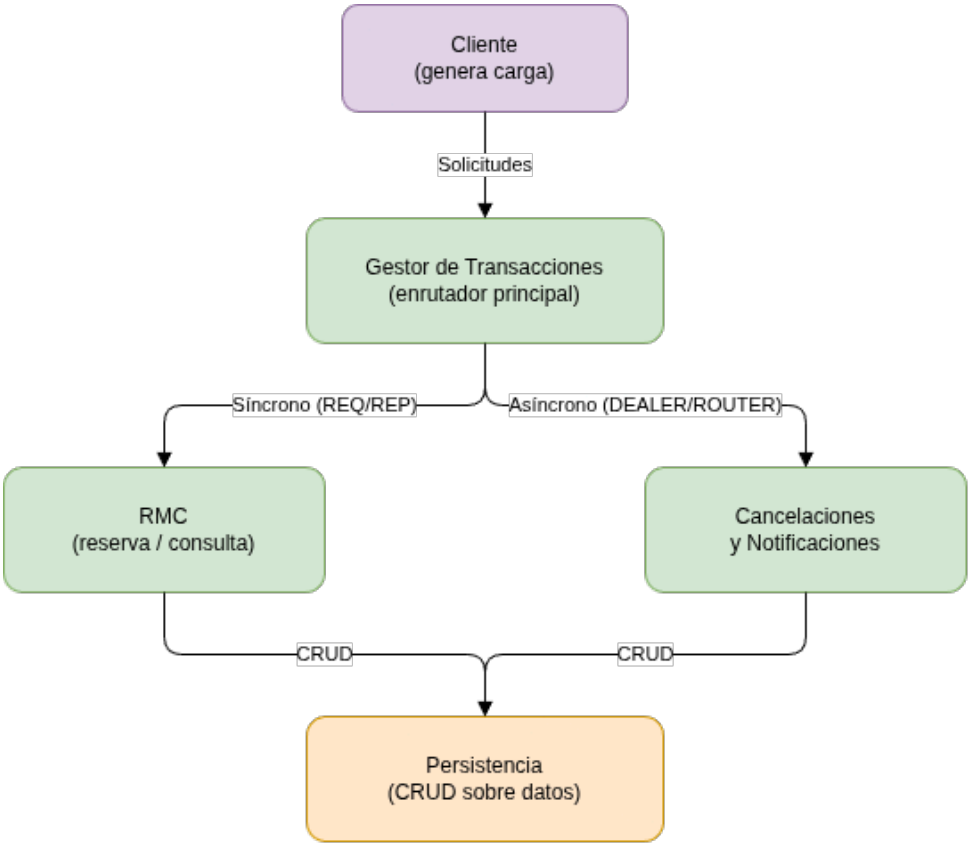


**Diagrama de Componentes**

El diagrama anterior muestra cómo cada componente se comunica según el tipo de operación. Las operaciones del sistema se componen de dos formas, dependiendo de cómo el cliente espera la respuesta:

**Operaciones síncronas**: hacen referencia a la consulta de eventos, reserva de puestos y modificación de una reserva. El cliente envía una solicitud y se queda en espera hasta recibir una respuesta antes de continuar. Se implementa con el patrón **REQ/REP** de ZeroMQ, tanto entre el Cliente y el Gestor de Transacciones como entre el Gestor de Transacciones y el servicio RMC.

**Operaciones asíncronas**: hacen referencia a la cancelación de una reserva. El cliente envía la solicitud y recibe de manera inmediata una confirmación de que fue recibida, sin tener que esperar a que la cancelación se complete. Cuando la cancelación finalmente se procesa en la base de datos, el sistema le avisa al cliente por separado mediante una notificación. Se implementa con **DEALER/ROUTER** para que el cliente no se bloquee al enviar la cancelación, y con **PUB/SUB** para el envío de la notificación.

Se asume un modelo de tiempo asíncrono: no hay garantías estrictas de límites de latencia entre nodos, dado que la comunicación ocurre sobre una red overlay (Tailscale) con latencias variables según la ubicación física real de cada dispositivo.

### Modelo de Fallos

El sistema se encarga de que si un componente falla, solo deja de responder, no se considera que un componente responda con información correcta o dañada intencionalmente. El Gestor de Transacciones y el servicio RMC, son los puntos donde puede ocurrir una falla, ya que estos son los responsables del flujo principal de la consulta y la reserva.

Para poder determinar si un componente falló, se usa un tiempo límite de espera. Si un proceso no recibe respuesta dentro de un lapso de tiempo razonable, asume que el otro extremo falló. También se considera la posibilidad de que un mensaje se pierda en el camino debido a la red, como puede ser una interrumpción temporal de la conexión, lo cual es distinto a que el proceso realmente haya fallado.

### Modelo de Seguridad

El sistema para esta primera entrega, no incluye mecanismos de seguridad avanzados, pero hay que recalcar los riesgos que son existentes. Los mensajes que viajan entre los servicios no van cifrados por ZeroMQ, pero al usar Tailscale, si quedan protegidos mientras viajan por la red, ya que esta cifra el tráfico automáticamente. También cualquier proceso conectado a la red Tailscale del equipo podría enviar solicitudes al sistema, ya que no hay ningina verificación de identidad de quién está haciendo la solicitud. Y por último, no se valida que los mensajes lleguen sin ser alterados, más allá de la protección que ya da la red. Estos mecanismos se aceptan dado que el etorno de pruebas es controlado. Como posible mejora futura, se podría agregar el mecanismo de CURVE de ZeroMQ para agregar autenticación de clientes y cifrado adicional.

## Diseño del Sistema

A continuación se presentan los modelos estructurales y de comportamiento del sistema:

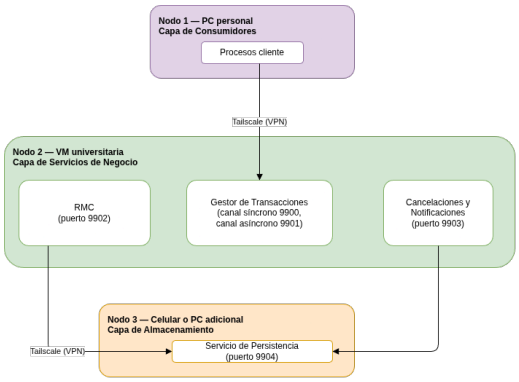



**Diagrama de Despliegue**

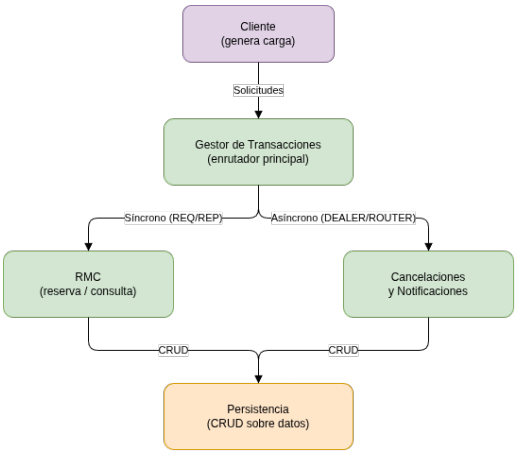



**Diagrama de Componentes**

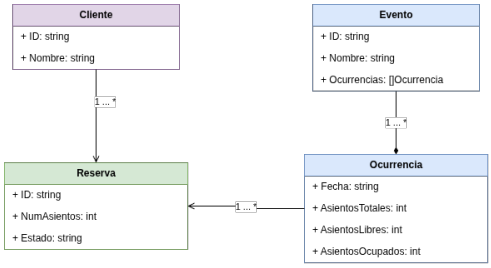



**Diagrama de Clases**

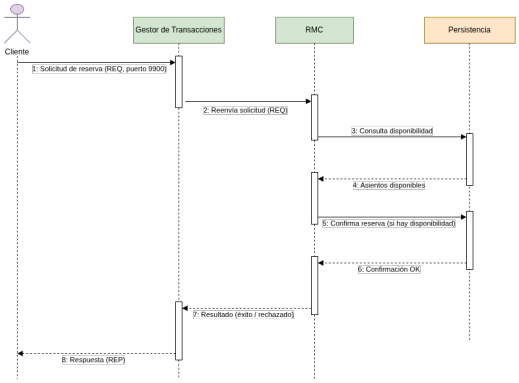



**Diagrama de Secuencia**

# Arquitectura Escalable y Tolerante a Fallos

# Protocolo de Pruebas

# Obtención de Métricas de Desempeño

# Generación de Carga